LAB 7: Fuzzing for Input Validation Bugs

Fuzz testing with the Hypothesis library

Name	P T Mounika
SRN	PES1UG24CS311
Assigned repo	SETAPESU26/L6_L7_06_event_ticket_tier
Working repo (fork)	github.com/mounika-200622/L6_L7_06_event_ticket_tier
Module under test	Lab 7/processor.py (event ticket tier)
Tools	Python 3.14, Hypothesis 6.170, pytest 9.1

Objective

Use property-based fuzz testing (Hypothesis) to find input validation bugs in the three functions of **processor.py** — *decode_promo_voucher*, *calculate_seat_tier_price* and *parse_attendee_badge* — and then harden the code so that no input can crash it.

Step 1: Installing dependencies

Installed **hypothesis** and **pytest** with pip.

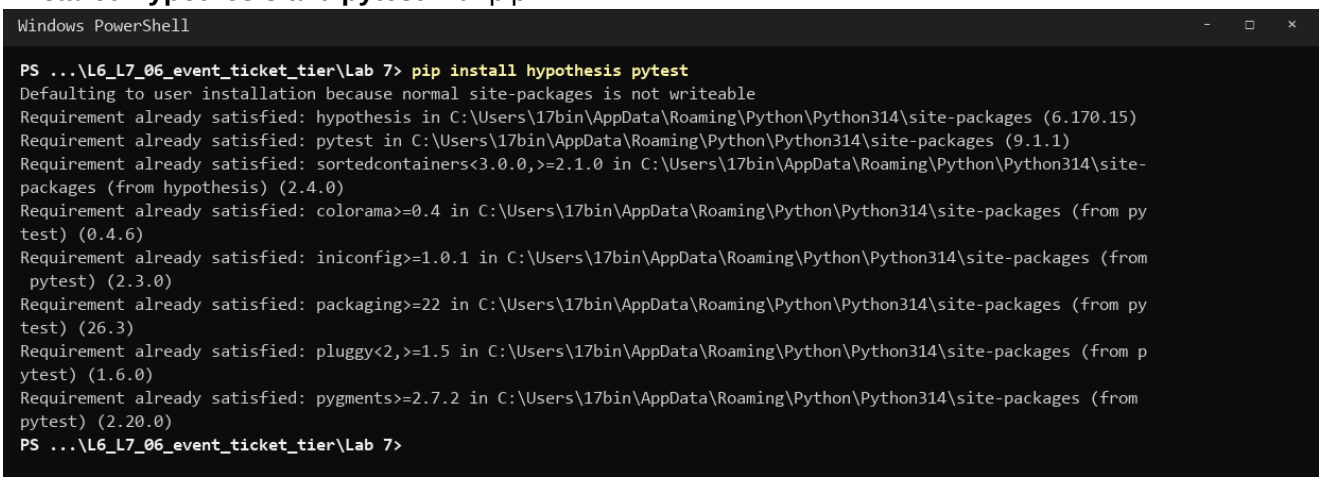


Fig 1: pip install hypothesis pytest

Step 2: Running the buggy application

With well-formed input the buggy script works. With a slightly malformed voucher (**EARLY20**, no dash) it crashes immediately with an unhandled **ValueError**.

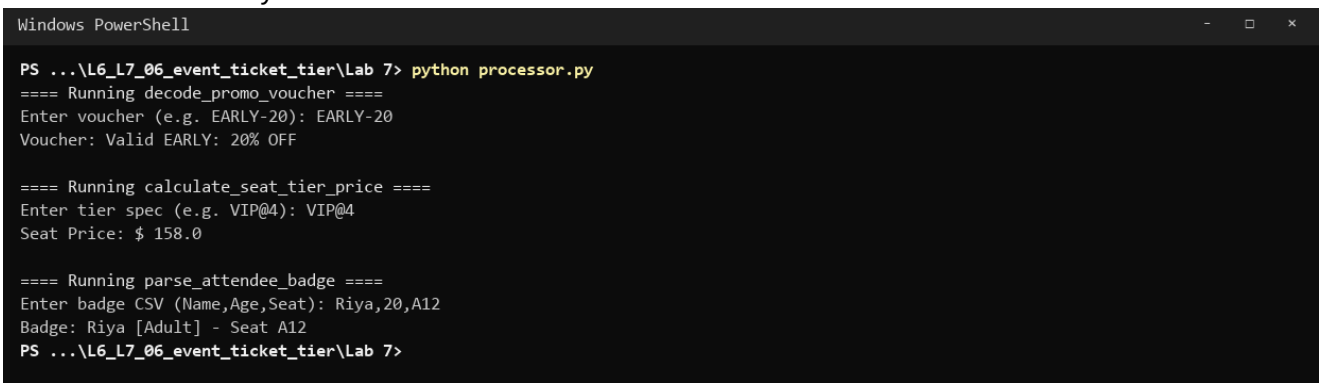


Fig 2a: buggy processor.py with valid input

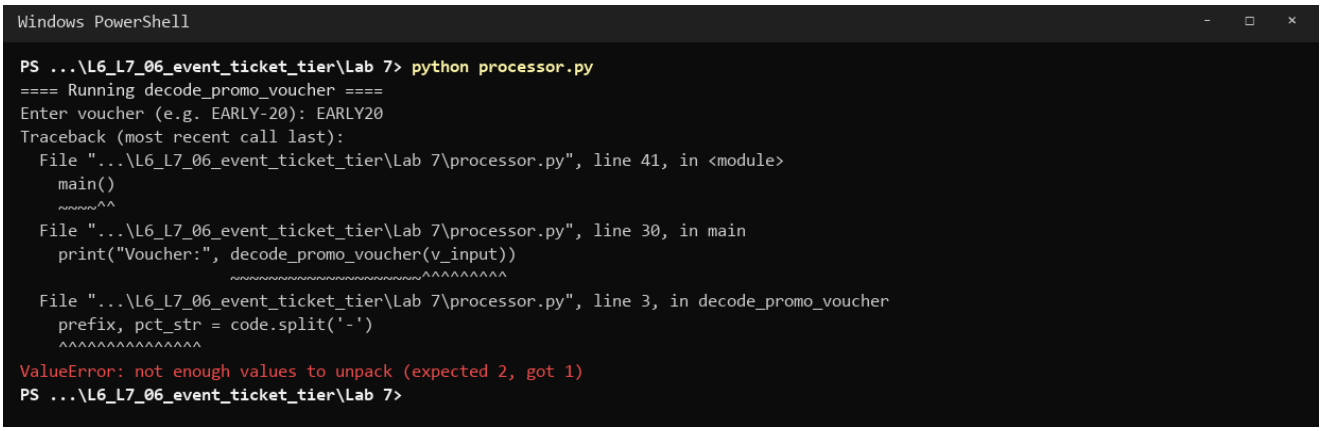


Fig 2b: buggy processor.py crashing on input 'EARLY20'

Step 3: Completing test_processor.py with Hypothesis

Each starter test was completed with a try/except block: the function is called with the generated input and any exception makes the test fail with the offending input. Besides *st.text()* | *st.none()*, custom strategies build "almost valid" inputs (e.g. *PREFIX-NUM*, *TIER@ROW*, *Name,Age,Seat* with random parts) so the fuzzer reaches deeper code paths. Three extra property tests check that valid inputs still produce the correct result after the fix. The full code is in the Appendix.

Step 4: Running the tests on the buggy code

All three fuzz tests fail. Hypothesis reports several distinct failures per function and shrinks each to the smallest failing input.

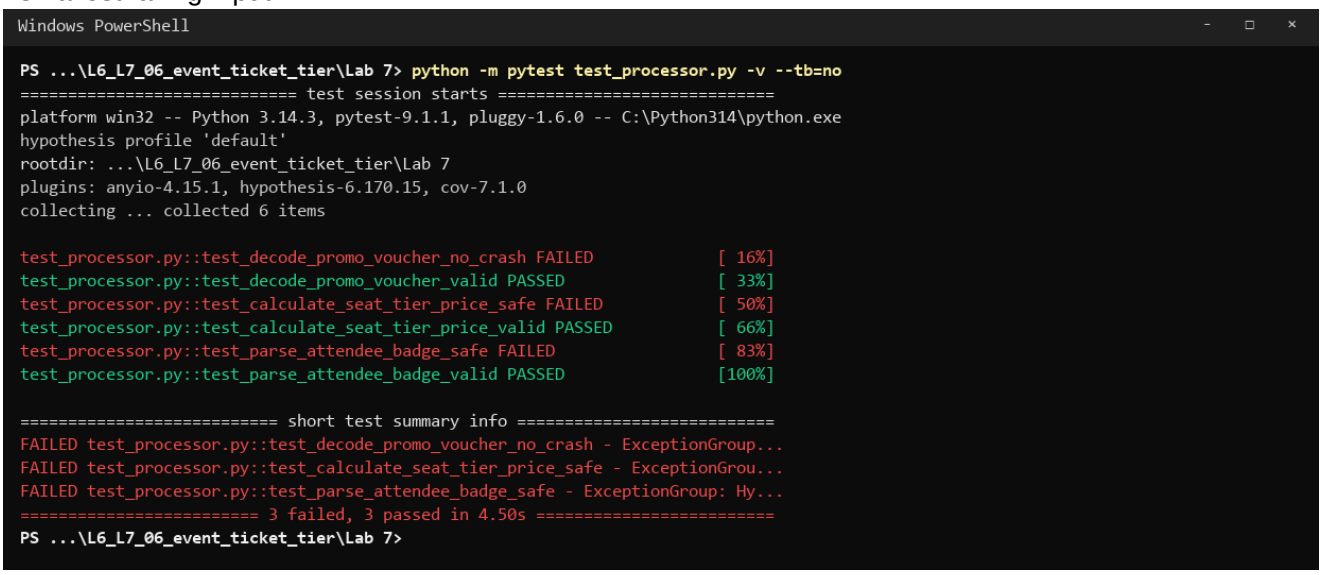


Fig 3: pytest - 3 fuzz tests FAILED on the buggy processor

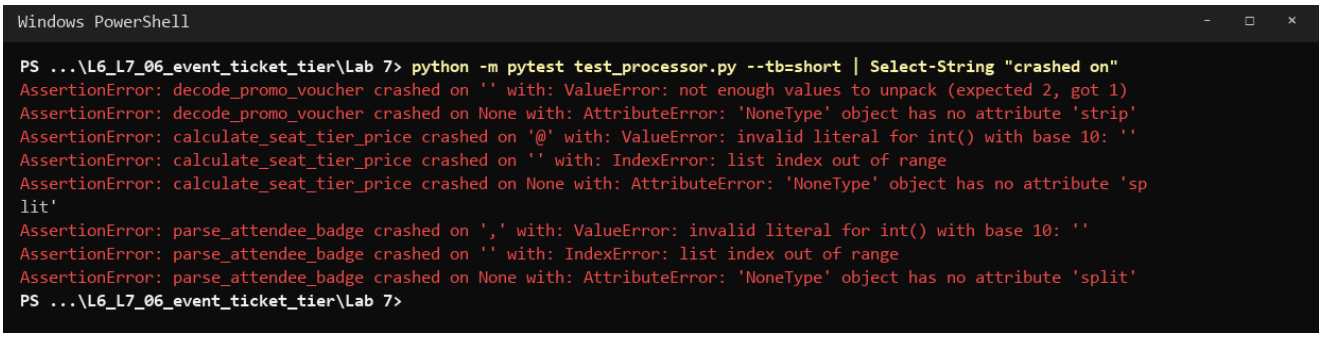


Fig 4: minimal failing inputs found by Hypothesis

Bugs found by fuzzing and how they were fixed

#	Function	Failing input	Error (buggy code)	Fix
1	decode_promo_voucher	None	AttributeError (.strip)	Type check: must be str
2	decode_promo_voucher	'' / 'EARLY20'	ValueError (unpack)	Require exactly 2 parts
3	decode_promo_voucher	'-'	ValueError int('')	Validate prefix and number
4	decode_promo_voucher	'X-500', 'X--5'	Accepted / crash	Percentage must be 1-100
5	calculate_seat_tier_price	None	AttributeError (.split)	Type check: must be str
6	calculate_seat_tier_price	''	IndexError parts[1]	Require exactly TIER@ROW
7	calculate_seat_tier_price	'@'	ValueError int('')	Row must be a whole number
8	calculate_seat_tier_price	'@0', 'GOLD@1'	KeyError (unknown tier)	Check tier in TIER_RATES
9	calculate_seat_tier_price	'VIP@-76'	Negative price (logic bug)	Row must be 1-1000
10	parse_attendee_badge	None	AttributeError (.split)	Type check: must be str
11	parse_attendee_badge	''	IndexError tokens[1]	Require exactly 3 fields
12	parse_attendee_badge	','	ValueError int('')	Age must be a whole number
13	parse_attendee_badge	',-5,'	Empty name / negative age	Name/seat required, age 0-130
14	all	very long strings	Possible DoS	Max input length 200

Step 5: Fixing the application

Every function now validates its input before using it: it checks the type (rejects None and non-strings), limits the length, checks the number of fields after splitting, converts numbers with a safe helper (*_to_int*, ASCII digits only), and checks ranges (percentage 1-100, known tier, row 1-1000, age 0-130). Instead of crashing, the function returns a clear **"Invalid ..."** message. *main()* also handles end-of-input safely.

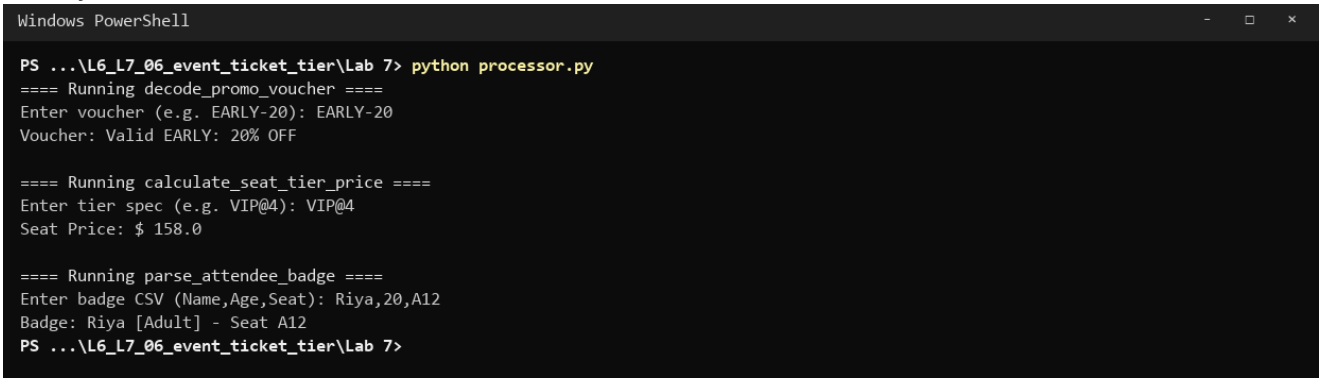


Fig 5a: fixed processor.py with valid input (same results as before)

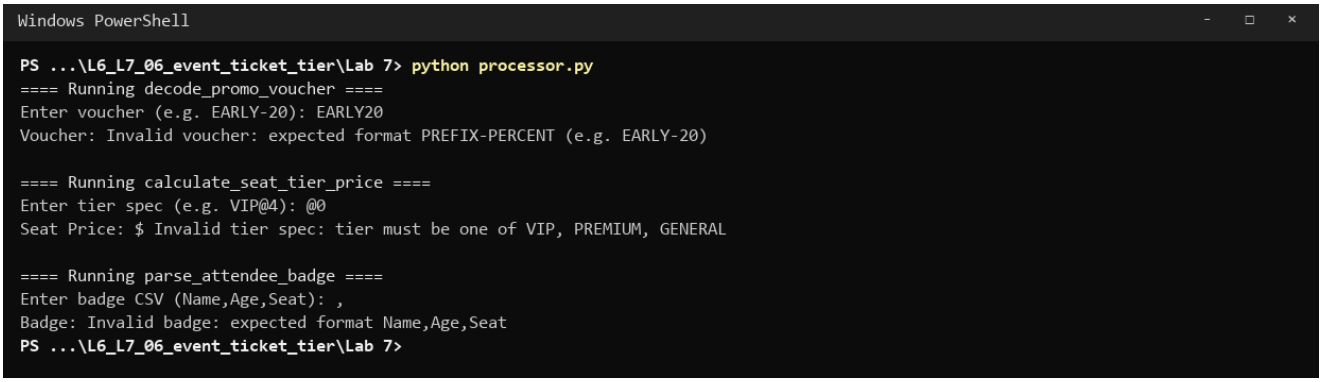


Fig 5b: fixed processor.py - the inputs that crashed it before now give error messages

Step 6: Re-running the tests on the fixed code

All 6 tests (3 fuzz tests + 3 valid-input property tests) pass.

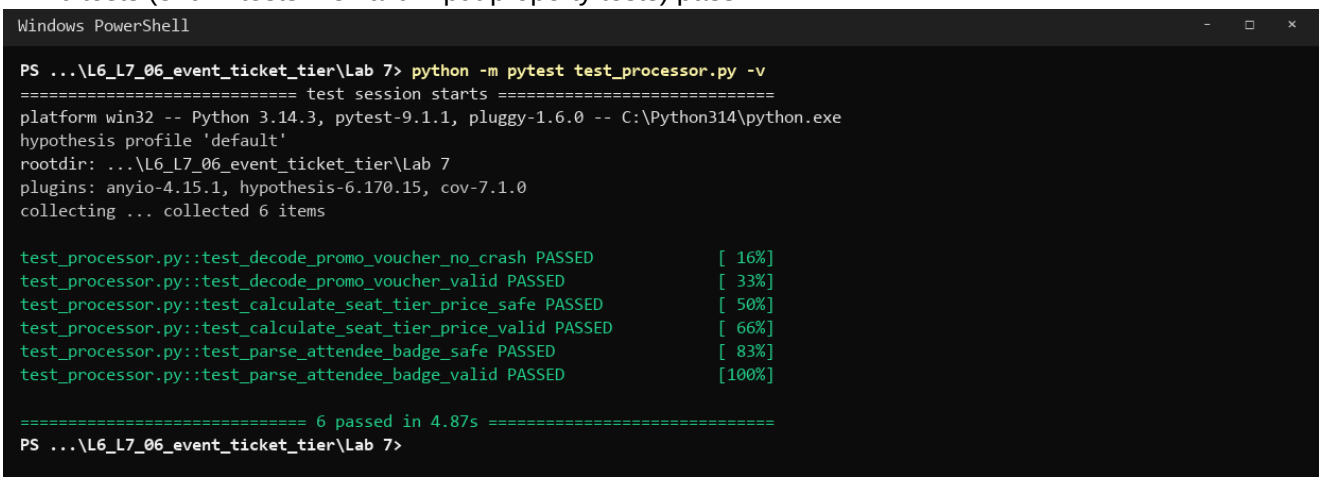


Fig 6: pytest - all 6 tests PASSED on the fixed processor

Conclusion

Hypothesis found 14 input validation problems that hand-written examples did not reveal: None values, empty strings, missing separators, unknown tiers, negative numbers and empty fields. After adding defensive validation, the fuzz tests pass across hundreds of random inputs per function, and valid inputs still produce the same results as the original code.

Appendix A: Fixed processor.py

```
"""
Lab 7 - Fuzz Testing with Hypothesis (FIXED processor)
Name : P T Mounika
SRN  : PES1UG24CS311
Repo : L6_L7_06_event_ticket_tier

Every function validates its input and returns an "Invalid ..." message
instead of raising an exception when it gets None, non-strings, empty
strings, missing/extra fields, non-numeric numbers or out-of-range values.
"""

TIER_RATES = {
    "VIP": 150.0,
    "PREMIUM": 90.0,
    "GENERAL": 45.0,
}

MAX_INPUT_LENGTH = 200   # guards against huge inputs (DoS-style)
MAX_ROW = 1000
MAX_AGE = 130


def _to_int(text):
    """Convert text to int only if it is a plain ASCII number, else None."""
    text = text.strip()
    sign = ""
    if text[:1] in ("+", "-"):
        sign, text = text[0], text[1:]
    if not text or not text.isascii() or not text.isdigit():
        return None
    return int(sign + text)


def decode_promo_voucher(voucher_code):
    if not isinstance(voucher_code, str):
        return "Invalid voucher: input must be a string"
    if len(voucher_code) > MAX_INPUT_LENGTH:
        return "Invalid voucher: input too long"

    code = voucher_code.strip()
    if not code:
        return "Invalid voucher: empty input"

    parts = code.split('-')
    if len(parts) != 2:
        return "Invalid voucher: expected format PREFIX-PERCENT (e.g. EARLY-20)"

    prefix, pct_str = parts[0].strip(), parts[1].strip()
    if not prefix or not prefix.isalnum():
        return "Invalid voucher: prefix must be letters/digits"

    percentage = _to_int(pct_str)
    if percentage is None:
        return "Invalid voucher: percentage must be a whole number"
    if not 1 <= percentage <= 100:
        return "Invalid voucher: percentage must be between 1 and 100"

    return f"Valid {prefix.upper()}: {percentage}% OFF"


def calculate_seat_tier_price(tier_spec):
    if not isinstance(tier_spec, str):
        return "Invalid tier spec: input must be a string"
    if len(tier_spec) > MAX_INPUT_LENGTH:
        return "Invalid tier spec: input too long"

    parts = tier_spec.split('@')
    if len(parts) != 2:
        return "Invalid tier spec: expected format TIER@ROW (e.g. VIP@4)"

    tier = parts[0].strip().upper()
    if tier not in TIER_RATES:
        return f"Invalid tier spec: tier must be one of {', '.join(TIER_RATES)}"

    row = _to_int(parts[1])
    if row is None:
        return "Invalid tier spec: row must be a whole number"
```

```
    if not 1 <= row <= MAX_ROW:
        return f"Invalid tier spec: row must be between 1 and {MAX_ROW}"

    base = TIER_RATES[tier]
    return base + (row * 2.0)


def parse_attendee_badge(badge_csv):
    if not isinstance(badge_csv, str):
        return "Invalid badge: input must be a string"
    if len(badge_csv) > MAX_INPUT_LENGTH:
        return "Invalid badge: input too long"

    tokens = badge_csv.split(',')
    if len(tokens) != 3:
        return "Invalid badge: expected format Name,Age,Seat"

    name = tokens[0].strip()
    seat = tokens[2].strip()
    if not name:
        return "Invalid badge: name is missing"
    if not seat:
        return "Invalid badge: seat is missing"

    age = _to_int(tokens[1])
    if age is None:
        return "Invalid badge: age must be a whole number"
    if not 0 <= age <= MAX_AGE:
        return f"Invalid badge: age must be between 0 and {MAX_AGE}"

    category = "Child" if age < 18 else "Adult"
    return f"{name} [{category}] - Seat {seat}"


def _safe_input(prompt):
    try:
        return input(prompt)
    except EOFError:
        return None


def main():
    print("==== Running decode_promo_voucher ====")
    v_input = _safe_input("Enter voucher (e.g. EARLY-20): ")
    print("Voucher:", decode_promo_voucher(v_input))

    print("\n==== Running calculate_seat_tier_price ====")
    t_input = _safe_input("Enter tier spec (e.g. VIP@4): ")
    print("Seat Price: $", calculate_seat_tier_price(t_input))

    print("\n==== Running parse_attendee_badge ====")
    b_input = _safe_input("Enter badge CSV (Name,Age,Seat): ")
    print("Badge:", parse_attendee_badge(b_input))


if __name__ == "__main__":
    main()
```

Appendix B: Updated test_processor.py

```
"""
Lab 7 - Fuzz Testing with Hypothesis
Name : P T Mounika
SRN  : PES1UG24CS311
Repo : L6_L7_06_event_ticket_tier

Property-based fuzz tests for processor.py.
Each function is fed random / malformed input (random text, None, and
"almost valid" structured strings) and must never raise an exception.
Extra property tests check that valid inputs still give the right result.
"""

from hypothesis import given, settings, strategies as st
from processor import decode_promo_voucher, calculate_seat_tier_price, parse_attendee_badge


# ---------------------------------------------------------------------------
# Strategies that build "almost valid" inputs so the fuzzer reaches deeper
# code paths than pure random text would.
# ---------------------------------------------------------------------------
voucher_like = st.builds(
    lambda prefix, sep, pct: f"{prefix}{sep}{pct}",
    st.text(max_size=10),
    st.sampled_from(["-", "--", "", " - ", "-x-"]),
    st.one_of(st.integers(), st.text(max_size=5)),
)

tier_like = st.builds(
    lambda tier, sep, row: f"{tier}{sep}{row}",
    st.one_of(st.sampled_from(["VIP", "premium", " General ", "GOLD", ""]), st.text(max_size=8)),
    st.sampled_from(["@", "@@", "", " @ "]),
    st.one_of(st.integers(), st.text(max_size=5)),
)

badge_like = st.builds(
    lambda name, age, seat, extra: ",".join([name, str(age), seat] + extra),
    st.text(max_size=10),
    st.one_of(st.integers(), st.text(max_size=4)),
    st.text(max_size=5),
    st.lists(st.text(max_size=3), max_size=2),
) | st.builds(lambda parts: ",".join(parts), st.lists(st.text(max_size=6), max_size=4))


# ---------------------------------------------------------------------------
# 1. decode_promo_voucher
# ---------------------------------------------------------------------------
@settings(max_examples=300)
@given(st.text() | st.none() | voucher_like)
def test_decode_promo_voucher_no_crash(s):
    try:
        result = decode_promo_voucher(s)
        assert isinstance(result, str)
    except Exception as e:
        assert False, f"decode_promo_voucher crashed on {s!r} with: {type(e).__name__}: {e}"


@given(st.from_regex(r"[A-Za-z]{1,10}", fullmatch=True), st.integers(min_value=1, max_value=100))
def test_decode_promo_voucher_valid(prefix, pct):
    assert decode_promo_voucher(f"{prefix}-{pct}") == f"Valid {prefix.upper()}: {pct}% OFF"


# ---------------------------------------------------------------------------
# 2. calculate_seat_tier_price
# ---------------------------------------------------------------------------
@settings(max_examples=300)
@given(st.text() | st.none() | tier_like)
def test_calculate_seat_tier_price_safe(s):
    try:
        result = calculate_seat_tier_price(s)
        assert isinstance(result, (float, str))
        if isinstance(result, float):
            assert result >= 0, "price must never be negative"
    except Exception as e:
        assert False, f"calculate_seat_tier_price crashed on {s!r} with: {type(e).__name__}: {e}"


@given(st.sampled_from([("VIP", 150.0), ("PREMIUM", 90.0), ("GENERAL", 45.0)]),
```

```
       st.integers(min_value=1, max_value=500))
def test_calculate_seat_tier_price_valid(tier_rate, row):
    tier, rate = tier_rate
    assert calculate_seat_tier_price(f"{tier.lower()}@{row}") == rate + row * 2.0


# ---------------------------------------------------------------------------
# 3. parse_attendee_badge
# ---------------------------------------------------------------------------
@settings(max_examples=300)
@given(st.text() | st.none() | badge_like)
def test_parse_attendee_badge_safe(s):
    try:
        result = parse_attendee_badge(s)
        assert isinstance(result, str)
    except Exception as e:
        assert False, f"parse_attendee_badge crashed on {s!r} with: {type(e).__name__}: {e}"


@given(st.from_regex(r"[A-Za-z]{1,12}", fullmatch=True),
       st.integers(min_value=0, max_value=120),
       st.from_regex(r"[A-Z][0-9]{1,3}", fullmatch=True))
def test_parse_attendee_badge_valid(name, age, seat):
    category = "Child" if age < 18 else "Adult"
    assert parse_attendee_badge(f"{name},{age},{seat}") == f"{name} [{category}] - Seat {seat}"
```